

Marmoset : A Compiler for Automatically Optimizing the Layout of Algebraic Data Types

ANONYMOUS AUTHOR(S)

While programmers know that how their data structures are laid out in memory can have significant effects on performance, compiler support to *optimize* that layout is an under-explored field. Prior work has either optimized the layout of individual structures without considering how collections of those objects in linked data structures might be laid out, or focused on arranging the placement of linked data structures without considering the internal layout of the constituent objects. This work introduces MARMOSSET, a compiler that optimizes the layouts of algebraic data types, with a special focus on producing highly optimized, *packed* data layouts where linked structures can be traversed with minimal pointer chasing.

MARMOSSET performs an analysis of how a recursive ADT is used in a function to choose how that ADT should be laid out in memory to promote simple, strided access through memory. It does so by building and solving a constraint system to minimize an abstract cost model, yielding a predicted efficient layout for the ADT. MARMOSSET then builds on top of GIBBON, a prior compiler for packed, mostly-serial representations, to synthesize optimized ADTs. We show experimentally that MARMOSSET is able to choose optimal layouts across a series of microbenchmarks and case studies, outperforming both GIBBON’s baseline approach, as well as the mature GHC compiler with its traditional, pointer-heavy, non-compact data representations.

1 INTRODUCTION

When programmers write code, they rely on compilers to make the low level decisions: which instructions to select, when instructions are scheduled to improve register and cache usage, where variables are placed on the stack, etc. But one area where programmers typically cannot rely on compiler support is in creating efficient data representations for complex data structures. While a compiler might tweak field placement in a structure to improve memory alignment, basic decisions like field ordering are typically chosen to simplify code generation rather than with an eye to efficiency. In high level languages, where data is “boxed”, elements in a structure may, indeed, be scattered throughout memory due to the decisions of runtime memory allocators. The problem is compounded with recursive, heap-allocated data structures: a program that simply traverses a linked list may ultimately hop randomly through memory, destroying cache locality and obviating the benefits of hardware prefetchers. The performance difference between a data layout that is well-matched to a program’s access patterns and one that is not can be vast.

Over the years, there has been some work exploring the space of more-intelligent data representations, tackling the problem from different angles. Some work targets memory management, making either memory allocation [Lattner and Adve 2002] or copying garbage collectors [Chilimbi and Larus 1998] sensitive to program behavior, allowing object locations to match access patterns. Other work modifies structure layouts to re-order fields and influence placement decisions [Chilimbi et al. 1999a,b; Truong et al. 1998]. However, these approaches only tackle one aspect of data representation—where objects are located—and primarily target non-recursive structures. Further, they fundamentally operate at the level of boxed data, requiring pointer chasing to access non-primitive data. When processing recursive, linked data structures, such approaches cannot (and do not attempt to) improve the efficiency of program by placing elements of data structures in such a way so as to improve traversal efficiency.

Recent work has tackled this problem from another direction: the Gibbon language and whole-program compiler [Vollmer et al. 2019, 2017] focuses on the problem of chasing pointers haphazardly

through memory when accessing recursive, linked structures. Gibbon produces *dense, packed* representations of algebraic data types where *unboxed* data is accessed directly, without pointer chasing. For example, tree data will be serialized in pre-order into a buffer, and traversing the tree (e.g., to sum all of its elements) will result in, essentially, a linear scan of an array, resulting in no pointer dereferencing and highly-efficient, predictable access patterns.

Gibbon has many benefits: programmers no longer need to take control of low-level data representation and allocation to serialize linked structures; and rather than writing error-prone index math to access data, the Gibbon compiler automatically translates idiomatic data structure accesses into operations on the serialized representation. However, Gibbon suffers from drawbacks complementary to the other approaches mentioned above. It does not attempt to match data layout to the access patterns of a program. If the particular serialization decisions for a data type chosen by Gibbon do not correspond to the behavior of functions accessing that data, then pathological behavior can result. Consider a tree laid out in left-to-right pre-order with a program that accesses that tree right-to-left. Rather than scanning straightforwardly through the structure, the program would have to jump back and forth through the buffer to access the necessary data. When encountering such cases, Gibbon does not break: instead, it inserts *shortcut pointers* to allow more-random access to structures [Vollmer et al. 2019]. But this defeats the purpose of a dense, packed representation: not only are accesses no longer nicely strided through memory, but the pointers and pointer-chasing of boxed data have returned. Indeed, when Gibbon is presented with a program whose access patterns do not match the chosen data layout, the generated code can be *significantly slower* than plain, pointer-based representations.

What we want is the best of both worlds: Gibbon’s dense, packed data layouts combined with access pattern-aware decisions of *how* to lay out that data. To that end, this paper introduces MARMOSET, an extension of the Gibbon compiler that creates dense, packed data representations that match the way a program accesses that data.

MARMOSET’s key insight is twofold. First, that a compile-time analysis can yield insights into how a program is *likely* to access the fields of a data structure, including recursive traversal patterns. Second, that Gibbon’s dense, serialized representation is ideal for taking advantage of this access information. Gibbon’s compilation strategy [Vollmer et al. 2019] makes explicit when field references will result in sub-optimal access patterns, making it possible to statically estimate the costs of different layout choices. This cost model can be used to drive a search process for a minimum-cost layout, subject to any additional constraints provided by the programmer.

MARMOSET thus analyzes the behavior of a program, and synthesizes a data layout that corresponds to that behavior. It then rewrites the data type and lowers the program to produce code that operates on a dense, packed data representation in a way that matches access patterns and data layout, resulting in improved locality.

We implement MARMOSET as an extension to the Gibbon compiler, including extending the source language to support (optional) layout hints. Thus MARMOSET can be either a transparent compiler optimization, or semi-automated tool for exploring different layouts during the programmers optimization work. One pleasant surprise is that MARMOSET need *not* be involved in the complicated mechanics of converting a program to operate on packed rather than boxed data (i.e., the location calculus [Vollmer et al. 2019]). Rather, in terms of phase ordering, MARMOSET does its work early in the compiler, after monomorphization, but before explicit regions-and-pointer-arithmetic serialization transformations. Thus the core language of MARMOSET (figure 2) is a simple first-order, monomorphic, strict, pure functional language.

Overall, in this paper:

- We provide a static analysis which generates a field access graph, capturing the temporal access patterns of a particular function on a particular ADT (algebraic datatype).
- Using this, together with a simple cost model, we formulate the field-ordering optimization problem as an Integer Linear Program and use a commercially available linear solver to get a mapping from fields to the optimal indices in the algebraic data type, consistent with any constraints provided by the programmer.
- We extend the Gibbon compiler to synthesize new datatypes based on the results of the optimization problem, and modify the program to use these new, optimized types.
- We show through our evaluation on a realistic blog software application and various other benchmarks that MARMOSSET can provide greater speedups than the best prior work, Gibbon. MARMOSSET also outperforms the mature, standard GHC compiler where boxed fields in ADTs is the default.

2 OVERVIEW

Functional languages, such as Haskell and dialects of ML, offer algebraic data types, a powerful language-based technology that can express many complex data structures and, nevertheless, provide a pleasantly high level of abstraction for application programmers. The high-level of specification of algebraic data types leaves a great deal of wiggle room for a range of low level implementation strategies. The way their structures are laid out in memory and how they are accessed by functions may have a major impact on performance. To see how, we are going to use a small program inspired by Pandoc, a popular document translation application written in Haskell. Specifically, in this case study, we use Pandoc’s document representation to build code for managing a blog. Our example program specifies the type `BlogList`, representing lists of blog posts.

```
type BlogList = [Blog]
data Blog = Blog Content HashTags
```

Representation of types, after monomorphization. Because this paper is concerned with the physical representation of types, we skip ahead to a point after the compiler frontend (monomorphizer) has specialized the list type to blogs. After monomorphization, we have the data type we are going to use for the rest of the paper:

```
data BlogList = Nil | Blog Content HashTags BlogList
```

Now, for a given non-empty blog value, we have a content field (a string representing the body of the blog post), a list of hash-tags summarizing keywords of the blog post, and a pointer to the rest of the list (i.e., the “cdr”). This monomorphized type is more efficient (even in a traditional *boxed* representation of algebraic datatypes) because the monomorphization inlines/unboxes the content and hashtags fields into each “cons cell”.

Mostly serialized data in GIBBON. In a conventional implementation of algebraic data types, consuming a value of a given algebraic data type, by default, requires consuming a full heap object, plus header word, plus a word-sized pointer. In GIBBON, the overhead of a data constructor is, by default, only a one-byte *tag* value, followed immediately by its fields. Wherever possible, that tag value occurs inline in a bytestream that hosts multiple values. As such, values tend to be compact in RAM and be laid out in contiguous blocks, which admit efficient linear traversals. When it has a (mostly) contiguous layout, a structure can be reliably traversed in a linear fashion, and thereby reap substantial benefits of modern optimizations such as hardware prefetching.

Traversal efficiency. Traversals of mostly serialized structures are *not necessarily* more efficient, however, because high efficiency requires that layout and access patterns line up. Let us consider the most favorable case in this regard: in our blog-management program, every function accessing

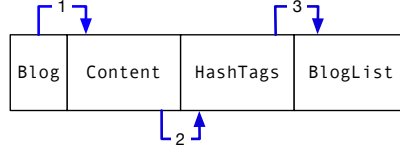
```

148 emphKeyword :: String → BlogList → BlogList
149 emphKeyword keyword blogs =
150   case blogs of
151     Nil → Nil
152     Blog content hashTags blogs' →
153       if search keyword hashTags then
154         let content' = emphContent content keyword
155             blogs'' = emphKeyword keyword blogs'
156         in Blog content' hashTags blogs''
157       else
158         let blogs'' = emphKeyword keyword blogs'
159         in Blog content hashTags blogs''

```

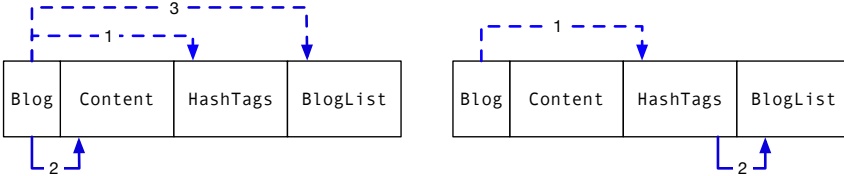
Fig. 1. Blog traversal motivating example

a blog value accesses the fields in the same order. Supposing that order of the accesses is the same as the order in which the fields appear in the type definition, then GIBBON can assign the following, compact (pointer-free) layout.



Blue arrows connecting adjacent fields represent efficient accesses—i.e., reading a range of bytes in a buffer, and then reading the next consecutive range. There is no work to compute the *start* of a downstream field, because its starting address is at the end of the preceding field!

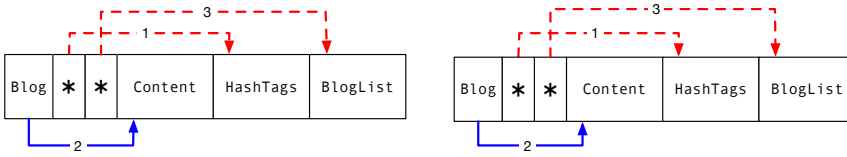
Skipping over fields of a data type. Often, however, access patterns are not so favorable. Let us consider our blog-processing program in figure 1. It traverses its given blog entries and puts an emphasis on all words that match a given search query. Importantly, the query depends on finding a given keyword in the hash-tags of the entry. When the query finds a match, that is, in the first branch of the conditional, the traversal accesses all the fields in the order, as depicted in the diagram on the left. In the other branch, the traversal skips over the content field, as depicted on the right. Here we use dotted lines to indicate accesses which must *skip over* some data between the current “cursor” and the target data.



The content field has a size that is not known statically, and therefore, without a pointer or size stored in the representation, the only way to skip over the field and get to the desired hash-tags field is to fully *traverse* the content field. This traversal is wasted work—an arbitrarily large amount.

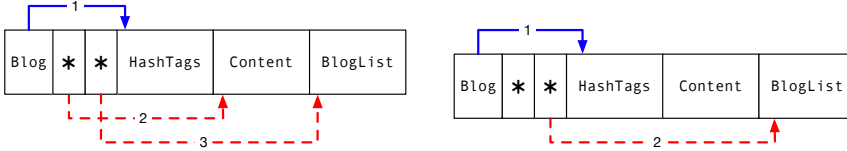
Shortcut pointers. In general, such “dummy” traversals can degrade the asymptotic efficiency of programs, and for this reason, GIBBON takes an alternative approach. When it detects the necessity

for the code to skip over intervening data, Gibbon changes the layout of the constructor by inserting *shortcut pointers*, which provide the size information needed to skip fields in constant time (similar in spirit to pointer swizzling). To handle our example program, Classic GIBBON would introduce one shortcut pointer for the hash-tags field, and another for the tail of the list. The latter is, in fact, not necessary, but introduced anyway because Classic GIBBON seeks to preserve asymptotic efficiency of programs, but lacks the sophisticated machinery to keep shortcut pointers to a minimum, in general. In our example program, we now see two shortcut pointers appearing just after the constructor tag, and we see the shortcut pointers being used to direct accesses to the hash-tags and list-tail fields (as opposed to taking their starting addresses from the results of traversing their preceding fields). We represent field access via shortcuts, storing either relative or absolute pointers: Crucially, the traversal in the “else” branch can now access the hash-tags field in constant time via the first shortcut pointer.



Yet, there remains plenty of room for improvement: if we optimize the data layout for this specific program, we can (1) further compact our blog items by doing with just one shortcut pointer and (2) achieve a better, more linear traversal in the “then” branch. To get these improvements, we need an approach that combines type-level and control-flow information and restructures the program accordingly.

Reordering fields for efficiency. Our approach is to increase compactness and linear access by taking into account temporal access patterns in the program. Given the results of a program analysis, we reorder the fields in type constructors (and adjust the code accordingly). Returning to our example, we can achieve our goals by swapping the ordering between the content and hash-tags fields. Given this reordering, our program’s “then” branch now proceeds in an efficient, linear pass through the blog structure, as desired. The access pattern in the “else” branch is also better, as it now visits the first field first, and, moreover, thanks to the shortcut pointer, it can skip over the content field in constant time.



In this example, we improved performance by reordering fields in the type, which is the only way we could have improved this program, owing to the data dependencies in the function. Indeed, this solution is the one produced automatically by our MARMOSSET extension of GIBBON. We note that, although our depictions above show two shortcut pointers, our MARMOSSET actually enables a significant compaction, thanks its reordering of the fields. With reordering, we can remove the first shortcut pointer, that is, the one pointing to the content field, because each of the two conditional branches in our program already fully traverse the hash-tags field, and thus already pay for finding the starting address of the content field. However, to realize this single-pointer version, there some improvements to the rest of the GIBBON compiler that are needed, and we leave such efforts to future work.

Customizable layouts. In MARMOSSET, we treat data ordering as a free variable, adapting it to the code that uses the data. We also provide annotations, which the user can add in comments to partially or fully constrain the search space of orderings (narrowing). There are also examples where widening the search space to include transformations of the *program order* as well would yield even better solutions, which could complement our approach, but are out of the scope of this paper.

From access patterns in the program to the field-access graph, a cost model of field orderings. The key to our approach is a cost model that assigns costs to different field orderings and a compiler pass that infers costs for given programs and assigns fields accordingly. Our cost model involves the inference for each data-type constructor in the program of a *field-access graph*. This graph represents the temporal ordering of accesses among fields and the associated costs, based on the control-flow graph of the program. Regarding the **Blog** constructor, for example, our analysis produces a field-access graph with three vertices representing the three fields, where each edge means that one field is accessed immediately before the other. The edges are induced by the two branches in our example function: in the “then” branch, we have an edge going from the hash-tags to the content field, and one from the content to the tail-pointer field; in the “else” branch there is an edge from the hash-tags to the list-tail field.

From field-access graph to optimized ordering, via ILP solver. From this graph, we obtain a cost model for different orderings by weighting each edge. Such weights are determined in our approach by approximate properties of the program’s control flow, e.g., the likelihood of a given access based on the branching structure leading to the access. Our approach takes a simple but effective approximation based on static control flow, but more sophisticated approximations are possible. Given a cost graph, MARMOSSET then encodes it as an optimization problem that can be solved by an off-the-shelf ILP (integer linear programming) solver. This optimization problem is tractable because its size is in proportion to the size of the data-type definition, not that of the whole program. From the resulting ILP solution, we can decode the result and obtain an efficient reordering of the fields. In general, our MARMOSSET analyses one pair of function definition and data-type constructor at a time, and rewrites the program to reflect the reordering of its fields.

3 DESIGN

MARMOSSET infers efficient layouts for densely packed data representations that match the way a program accesses that data. As described in Section 2, this essentially boils down to inferring an efficient ordering for fields of data constructors such that they align with the order those fields are accessed in a function. MARMOSSET operates on the language λ_M given in figure 2. λ_M is a first-order, monomorphic, call-by-value functional language with algebraic datatypes and pattern matching. Programs consist of a series of datatype definitions, function definitions, and a main expression. λ_M ’s expressions use A-normal form [Flanagan et al. 1993]. The notation $\bar{x}$ denotes a vector $[x_1, \dots, x_n]$ and $\bar{x}_i$ the item at position i . λ_M is an intermediate representation (IR) used towards the front end in the Gibbon compiler. The monomorphizor and specializer lower a program written in polymorphic, higher-order subset of Haskell¹ to λ_M , and then location inference is used to convert it to the *location calculus* (LoCal) code next. It’s easier to update the layout of types in λ_M compared to LoCal, since in λ_M the layout is implicitly determined by the ordering of fields, whereas the later LoCal IR makes the layout explicit using locations and regions (essentially, buffers and pointer arithmetic). In the remainder of this section, we describe the components of MARMOSSET’s design in turn.

¹With strict evaluation semantics using -XStrict.

$K \in$ Data Constructors, $\tau \in$ Type Constructors, $x, y, f \in$ Variables

Top-Level Programs	top	$::= \overline{dd}; \overline{fd}; e$
Field of a Data Constructor	fe	$::= \tau$
Datatype Declarations	dd	$::= \text{data } \tau = \overline{K \overline{fe}}$
Function Declarations	fd	$::= f : \overline{\tau} \rightarrow \tau; f \overline{x} = e$
Values	v	$::= x$
Expressions	e	$::= v$
		$ f \overline{v}$
		$ K \overline{v}$
		$ \text{let } x : \tau = e \text{ in } e$
Pattern	pat	$ \text{case } v \text{ of } \overline{pat}$
		$::= K (\overline{x : \tau}) \rightarrow e$

Fig. 2. Simplified grammar for a first-order, monomorphic functional language that MARMOSSET operates on. The full implementation includes more primitive datatypes, and built in primitives (such as a copy primitive).

3.1 Control Flow Analysis

The control-flow graph (CFG) in traditional compilers such as LLVM is made up of nodes representing *basic blocks of effectful instructions* in the intermediate language and edges between basic blocks represent the control flow of the program, that is, jump instructions which cause the program to transfer control from one block to another. λ_M is a functional language and therefore does not have an analogous abstraction. We construct a CFG with flattened, let-bound expressions of λ_M as the nodes.

Given a function definition, each sub-expression of the function body becomes a separate node in the CFG. An edge between two nodes indicates the order of the evaluation of the program². A node corresponding to a **let**-binding contains the bound expression and has one outgoing edge to a node corresponding to the body expression. A **case** expression splits the control flow n -ways, where n is the number of pattern matches. Outgoing edges of a node for a **case** expression have weights associated with them, which correspond to the likelihood of taking a particular branch in the program. If available, dynamic profiling information can provide these weights. For the purposes of this paper, however, we find it is sufficient to use trivial placeholder weights: i.e. uniform weight over all branches. A node for a pattern match clause merely has an outgoing edge to its body expression. Control flow terminates on either a data constructor, a function application, or a variable reference.

In the current implementation, a function call *does not* correspond to an edge to the callee's body in the CFG, since MARMOSSET only analyses one function at a time. In the future, we plan to extend our work to perform interprocedural analysis. However, in the whole-program compilation setting (where types are already specialized and monomorphic), non-recursive functions can very often be inlined yielding a data traversal as a single recursive function.

Figure 3 shows the control-flow graph for the running example from figure 1. Each node corresponds to a sub-expression of the function `emphKeyword`. The first **case** expression splits the control flow into two branches, corresponding to whether the input list of blogs is empty or not. The branch corresponding to the empty input list is assigned a probability α , and thus, the

²The order of evaluation is straightforward to determine since λ_M uses call-by-value semantics.

other branch is assigned a probability $1 - \alpha$. The next node corresponds to the pattern match Blog content hashTags blogs'. Another two-way branch follows, corresponding to whether keyword occurs in the content of this blog or not. We assign the probabilities σ and $1 - \sigma$ to these branches respectively. Each of these branches terminate by creating a new blog entry with its content potentially updated. In the current model, α and σ are 0.5: they are uniformly distributed. One intuition for why realistic branch weights are not essential to MARMOSSET's optimization is that accurate weights only matter if we're *trading off* between control-flow paths that are best served by different layouts. The base cases (e.g. empty list) typically contribute *no* ordering constraints, and the algorithms we've studied so far each have a preferred-order per-function, not tradeoffs intra-function, which would reward having accurate, profile-driven branch probabilities.

3.2 Field Access Pattern Analysis

After constructing the CFG for a function definition, we inspect the type of each input parameter—one data constructor at a time—and construct a *field-access graph* for it. This graph represents the temporal ordering of accesses among its fields according to the function's CFG. The fields of the data constructor form the nodes of this graph. A directed edge from field f_i to field f_j is added if f_i is accessed immediately before f_j . Each edge has a associated weight which indicates the likelihood of accessing f_i before f_j , which is computed using the CFG.

The field-access graph (subsequently referred to as G) is a directed graph which is allowed to have cycles. The directed nature of the edges enforces a temporal relation between the two fields it connects. More concretely, let there be an edge e' that connects two vertices or fields fe_a and fe_b . Assuming the head of the edge e' starts at fe_a and the tail meets fe_b , we interpret this as a temporal access such that field fe_a is accessed right before field fe_b , and vice-versa. The weight w' for the edge e' is the probability that this access will happen based on statically analyzing the function fd .

In our analysis, for a unique path through the traversal, we only account for the first access to any two fields. If two fields are accessed in a different order later on, the assumption is that the start address of the fields is likely to be in cache and hence it does not incur an expensive fetch call to memory. In fact, we tested our hypothesis by artificially making an example where say field f_a is accessed first, field f_b is accessed after f_a after which we constructed multiple artificial access patterns from f_b to f_a . This would suggest placing f_b before f_a . However, once the cache got warmed up and start addresses of f_a and f_b are already in cache, the layout did not matter as much. This suggests that prioritizing for the first access edge between two unique fields along a unique path is sufficient for our analysis.

Two fields can be accessed in a different order along different paths through a traversal. This results in two edges between the fields (The edges are in reversed order). We allow a max of two edges between any two vertices with the constraint that they have to be in the opposite direction and come from different paths in the traversal. If two fields are accessed in the same order along different paths in the traversal, we simply add the probabilities and merge the edges since they are in the same direction.

In order to construct G , we topologically sort the control flow graph of the function fd and traverse it in a breath first (*bfs*) fashion. We maintain an environment that tracks aliases to fields in the data constructor for which we are constructing G . As we process each node (i.e. a primitive expression such as a single function call), we update the graph for any direct or indirect references to input fields that we can detect. We ignore new variable bindings that refer to newly allocated rather than input data—they are not tracked in the access graph. We traverse the control-flow graph once, but we maintain the last-accessed information at each CFG node, so when we process a field access at an expression, we consult what was previously-accessed at the unique predecessor of

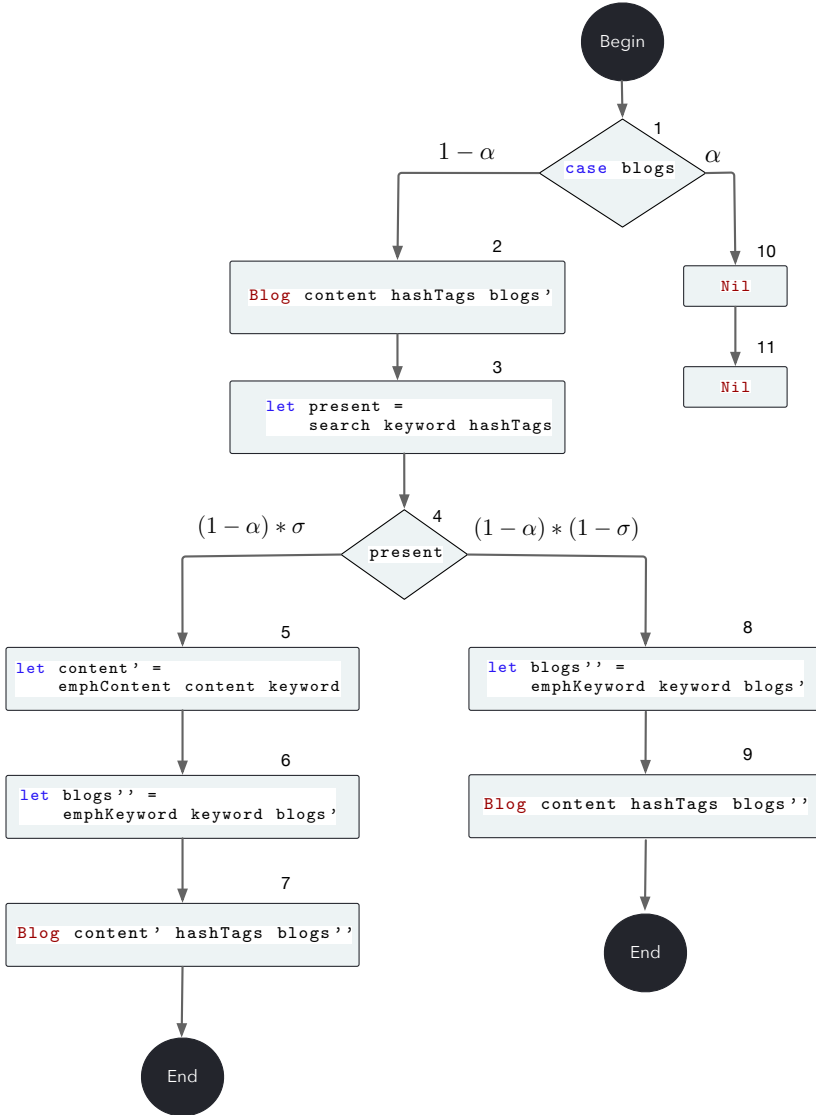


Fig. 3. Control flow graph generated from function body of running example 1.

the current CFG node. Figure 4 shows the generated access graph from the control flow graph in figure 3. It also shows the probability along each edge obtained from the control flow graph.

As we are traversing the nodes of the control flow graph and generating directed edges in G , we also keep track of the weights for each edge and store the weights appropriately. As mentioned previously, we can get more precise weights by dynamically profiling the traversal at runtime to obtain the frequency of taking a particular path through the traversal. However, for now, we restrict ourselves to static weights.

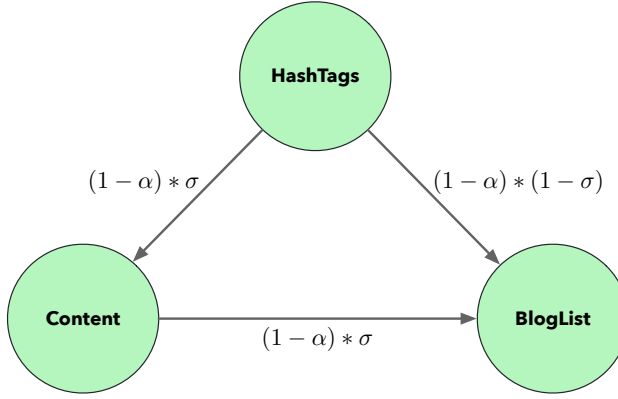


Fig. 4. Field access graph G generated from CFG of running example 3. Here we use field types to uniquely name them, since Haskell record syntax was not used in this example to give each field a name.

3.3 Finding a Layout

We use the field-access graph G to formulate an Integer Linear Program (ILP) that computes a cost-optimal field order for the data constructor and function pair.

3.3.1 ILP Constraints. In our encoding, each field in the data constructor is represented by a variable, $f_0, f_1, \dots$, that is assigned a unique integer in the interval $[0, \text{length}(\text{fields}) - 1]$. Intuitively, think of the variables as representing a positional index in the sequence of fields. Thus, the ILP uses the following constraints:

$$f_i \in \mathbb{W}$$

$$\forall_{0 \leq i < n} \quad 0 \leq f_i < n \quad (1)$$

$$\forall_{0 \leq i < j < n} \quad f_i \neq f_j \quad (2)$$

The constraints of form 1 ensure that each field is mapped to a valid index, while the constraints of form 2 ensure that each field has a unique index. These constraints represent *hard* constraints on the layout: each field needs to be in a valid location. However, MARMOSSET's goal is to find layouts that improve efficiency. To this end, what we are concerned with are *soft* constraints: based on the access pattern of a function, we would *prefer*, for example, that field a be before field b , but it will not harm correctness if that constraint were violated.

3.3.2 COST MODEL. MARMOSSET encodes these soft constraints in the form of an abstract cost model that assigns a cost to a given layout (assignment of fields to positions) based on how efficient it is expected to be given the field-access graph.

To understand the intuition behind the cost model, note that the existence of an edge from field f_i to field f_j in the field-access graph means that there exists at least one path in the control-flow graph where f_i is accessed and f_j is the next field of the data constructor that is accessed. In other words, the existence of such an edge implies a preference *for that control flow path* for field f_i to be immediately before field f_j in the layout so that the program can continue a linear scan through the packed buffer. Failing that, it would be preferable for f_j to be “ahead” of f_i in the layout so the

program does not have to backtrack in the buffer. We can thus consider the costs of the different layout possibilities of f_i and f_j :

C_{succ} (f_j immediately after f_i): This is the best case scenario: the program traverses f_i and then uses f_j .

C_{after} (f_j after f_i in the buffer): If f_j is after f_i , but not *immediately* after, then the code can proceed without backtracking through the buffer, but the intervening data means that either a shortcut pointer or a dummy traversal must be used to reach f_j , adding overhead.

C_{pred} (f_j immediately before f_i): Here, f_j is *earlier* than f_i in the buffer. Thus, the program will have already skipped past f_j , and some backtracking will be necessary to reach it. This incurs *two* sources of overhead: skipping past f_j in the first place, and then backtracking to reach it again.

C_{before} (f_j before f_i in the buffer): If, instead, f_j is farther back in the buffer than f_i , then the cost of skipping back and forth is greater: in addition to the costs of pointer dereferencing, because the fields are far apart in the buffer, it is less likely f_j will have remained in cache (due to poorer spatial locality).

We note a few things. First, the *exact* values of each of these costs are hard to predict. The exact penalty a program would pay for jumping ahead or backtracking depends on a variety of factors such as cache sizes, number of registers, cache line sizes, etc. Second, one could imagine a more fine-grained cost model that distinguishes between, say, fields that are a constant distance apart versus those that have some other recursive (and hence potentially large) field between them. However, the inexact nature of the cost model means that this would be false precision. Instead, MARMOSET uses an abstract cost model with reasonably-chosen values satisfying the following:

$$C_{succ} < C_{after} < C_{pred} < C_{before} \quad (3)$$

3.3.3 ASSIGNING COSTS TO EDGES. MARMOSET uses the field-access graph and this cost model to construct an objective function for the ILP. We note that each edge in the access graph represents one pair of field accesses with a preferred order. Thus, for each edge $e = (i, j)$, MARMOSET can use the indices of the fields f_i and f_j to assign a cost, c_e , to that pair of accesses, were it to happen.

If f_j is right after f_i , assign cost C_{succ} :

$$(f_j - f_i) = 1 \rightarrow c_e = C_{succ}$$

If f_j is farther ahead of f_i , assign cost C_{after} :

$$(f_j - f_i) > 1 \rightarrow c_e = C_{after}$$

If f_j is immediately *before* f_i , assign cost C_{pred} :

$$(f_j - f_i) = -1 \rightarrow c_e = C_{pred}$$

And if f_j is farther before f_i , assign cost C_{before} :

$$(f_j - f_i) < -1 \rightarrow c_e = C_{before}$$

The cost of each edge, c_e must be multiplied by the *likelihood* of that edge being exercised, p_e , which is also captured by the field-access graph, in the form of edge weights. Combining these gives us a total estimated cost for any particular field layout:

$$C = \sum_{e \in E} c_e \cdot p_e \quad (4)$$

This is the cost that our ILP attempts to minimize, subject to constraints 1 and 2.

4 IMPLEMENTATION DETAILS

We implement MARMOSSET in the open-source Gibbon compiler³. Gibbon is a whole-program micropass compiler that compiles a polymorphic, higher-order subset of (strict) Haskell. The Gibbon front-end uses standard whole-program compilation and monomorphization techniques [Chlipala 2015] to lower input programs into a first-order, monomorphic IR (λ_M). Gibbon performs location inference on this IR to convert it into a LoCal program, which has regions and locations, essentially, buffers and pointer arithmetic. Then a big middle section of the compiler is a series of LoCal- $\rightarrow$ LoCal compiler passes that perform various transformations. Finally, it generates C code. Our extension operates towards the front-end of the compiler, on λ_M . We closely follow the design described in Section 3 to construct the control-flow graph and field-access graph, and use the standard Haskell graph library⁴ in our implementation.

4.1 Solving Cost Model Constraints

We construct the field access graph and ILP (integer linear program) as detailed in Section 3. The abstract cost model works with four costs, subject to the constraints in Equation 3. Because every term of the objective function (Equation 4) is a multiple of one of these costs, the only thing that matters is the relative values of the costs, not their absolute value. Hence, we use a ratio of 1:2:3:4 for C_{succ} , C_{after} , C_{pred} and C_{before} , respectively.

After constructing the ILP, we use a commercially available ILP solver, CPLEX [cpl 2023], to obtain a minimum-cost ordering. The logical constraints described in section 3.3 cannot directly be given as an input to CPLEX. To solve these constraints, we use IBM's DOCPLEX (Decision Optimization CPLEX), because its API allows high level modelling such as logical expressions like implications, negations, logical AND etc. with relatively low overhead. Unfortunately, there isn't a readily available Haskell library that can interface with DOCPLEX. Thus, we use it via its library bindings available for Python. Specifically, we generate a Python program that feeds the constraints to DOCPLEX and outputs an optimum field ordering to the standard output, which MARMOSSET reads and parses, and then reorders the fields accordingly. The field ordering is output as a list of tuples, $[(Int, Int)]$, where the first component is the original field index and the second component is that field's new index.

4.2 Reordering Fields of a Datatype

Once we obtain an optimum field ordering for a specific data constructor in the context of a function definition, we update that data constructor to have this new field order. Consequently, we need to repair the complete program to adhere to the updated data constructor.

4.3 Annotation Pragmas for Optional, Manual Ordering Constraints

Optional, manual control over data layout has uses beyond second guessing the automatic optimizer. For instance, the user may wish to store the data in a certain format on disk or provide information for ordering that may affect performance downstream for reasons the user has information about. In addition, the user may want to read existing data in a certain format residing on disk. We allow the user to specify the layout of fields in MARMOSSET through annotations (pragmas). Currently, our compiler allows these specifications to be defined for a specific function and data constructor[s]. We allow the user to specify three kinds of constraints over the fields of a data constructor. These are shown in figure 5.

³<https://github.com/iu-parfunc/gibbon/>

⁴<https://hackage.haskell.org/package/containers>

```

-- User defines some absolute constraints as assignments of field indices.
{-# ANN Cons (foo, (0 ~> 3, 1 ~> 2)) #-}

-- User defines some relative ("after") constraints.
{-# ANN Cons (foo, (4 :> 5, 3 :> 2)) #-}

-- User requests the compiler optimize the Cons constructor
-- based on the access patterns of fields in traversal foo.
{-# ANN foo Cons #-}
data List = Cons F0 F1 F2 F3 F4 F5 List
          | Nil

foo :: List → List
foo lst = case lst of
           Cons f1 f2 f3 f4 f5 f6 rst → ...

```

Fig. 5. User annotations in MARMOSET to define constraints over fields of a data constructor. The annotations are defined as ANN annotations for a specific function and data constructor that function uses.

- **Absolute Constraints:** These constraints allow the user to specify an exact position for a field in a data constructor. The ANN annotation names a specific data constructor, a function name, and a list of field (indices) as well as their assigned positions. Hence $0 \sim 3$ in the ANN annotation can be assigned the field at position 0 to position 3 in the data constructor. This is parsed by the compiler and added as metadata in the function body. The constraint solver honors these constraints, while freely moving around the other fields. For example, $0 \sim 3$ will generate the following constraints in the constraint solver : $F0_{index} = 3$
- **Relative Constraints:** These constraints allow the user to specify the relative ordering of fields in a data constructor. Currently we support “Immediately After” constraints but these can be extended to other types of relative constraints in the future. Immediately After constraints allow the user to place one field right after another one. For instance, “ $4 :> 5$ ” can be interpreted as placing field at index position 4 right after the field at index position 5. Such a specification by the user will generate the following constraint in the constraint solver : $F4_{index} = F5_{index} + 1$ for the example shown in figure 5.
- **Optimization Hints:** The user can specifically invoke MARMOSET on a particular combination of function and data constructor(s). This is a hint from MARMOSET to process the function, and may be subsumed by future strategies that automatically apply (and cache solver results for) the optimization. In any case, the performance optimization search previously described will always respect any other hard constraints that have been annotated.

5 EVALUATION

We evaluate MARMOSET through a realistic blog software application as well as microbenchmarks. The microbenchmarks include traversals over lists, binary trees and a toy logical expression evaluator. The blog software application is written in the GIBBON language which is a subset of Haskell. This application has a data type for a Blog which is the major data structure holding the various components of a Blog. The Blog data type makes use of multiple simplified *Pandoc*⁵ data structures to represent the content in our blog software.

⁵Pandoc library: (<https://pandoc.org>)

	<i>List</i>	<i>List'</i>	MARMOSSET
<i>MeasureLength</i>			
<i>Mean</i>	59.62ms	1.384ms	1.887ms
<i>Median</i>	59.58ms	1.385ms	1.879ms
<i>CI 95%</i>			
<i>ub</i>	59.696ms	1.385ms	1.871ms
<i>lb</i>	59.551ms	1.383ms	1.902ms

Table 1. Runtimes (9 runs) : Mean, Median and 95% confidence interval in milli seconds(ms) for different layouts when compiled with GIBBON. The last column shows the runtime for the layout chosen by MARMOSSET.

We use GIBBON as the baseline for our benchmarks and evaluate the performance of MARMOSSET compared to GIBBON (which is the most closely related prior work). For each benchmark, we run the benchmark for 9 iterations and report the median runtime of the traversal. We also show the performance of MARMOSSET compared to the mature GHC system.

5.1 Experimental Setup

We run our benchmarks on a server-class machine with 64 CPUs, each with two threads. The CPU model is AMD Ryzen Threadripper 3990X with 2.2 GHz clock speed. The L1 cache size is 32 KB, L2 cache size is 512 KB and L3 cache size is 16 MB. We compile our programs with GCC version 7.5.0 and use the `-O3` optimization option.

5.2 Micro Benchmarks

We use three microbenchmarks to explore specific performance penalties imposed by a sub-optimal data layout:

5.2.1 Computing the length of a linked-list. This benchmark demonstrates the cost of de-referencing memory addresses which are not present in the cache. It uses a linked list datatype, with inlined variable-sized content:

```
data List = Nil | Cons Content List
```

If each element of the list is constructed using `Cons`, the traversal has to de-reference a pointer—to *jump over* the content—each time to access the tail of the list. This is an expensive operation, especially if the target memory addresses is not present in the cache. On the contrary, if the `Content` and `List` fields were swapped, then to compute the length, the program only has to traverse n bytes for a list of length n —one byte per `Cons` tag—which is extremely efficient. Essentially, MARMOSSET transforms program to use the following datatype, while preserving its behavior:

```
data List' = Nil | Cons List' Content
```

In our experiment, the linked-list is made up of 3M elements and each element contains some content (a list of strings) that occupies roughly 5KB. As seen in table 1, the performance of the list constructed using the original `Cons` is $\sim 43\times$ worse than the performance with the MARMOSSET-optimized, flipped layout. Not only does the `Cons`-list have poor data locality and cache behavior, but it also has to execute more instructions to de-reference the pointer.

5.2.2 Short-circuiting logical And/Or operators. These microbenchmarks operate over a synthetically generated, balanced syntax-tree with height a height of 45. The intermediate nodes can be one of `Not`, `Or`, or `And`, selected at random, and the leaves consist of booleans wrapped in a `Val`.

	<i>lr</i>	<i>rl</i>	MARMOSET
<i>LogicalExpressionEvaluator</i>			
<i>Mean</i>	$7.98e-3$	$1.38e-2$	$7.97e-3$
<i>Median</i>	$7.95e-3$	$1.38e-2$	$7.94e-3$
<i>CI 95%</i>			
<i>ub</i>	$8.01e-3$	$1.38e-2$	$7.99e-3$
<i>lb</i>	$7.95e-3$	$1.38e-2$	$7.94e-3$

Table 2. Runtimes : Mean, Median and 95% confidence interval in seconds for different layouts when compiled with GIBBON. The last column shows the runtime for the layout chosen by MARMOSET. Legend: l - Left subtree, r - Right subtree of the decision trees that are the main data structure of the logical expression evaluator.

The evaluator function evaluates the expression tree and performs short circuiting for expressions like `and` and `or` in the expression tree. We measure the performance of this evaluator function by changing the order of the left and right subtrees. The syntax-tree datatype is defined as follows:

```
data Exp = Val Bool | Not Exp | Or Exp Exp | And Exp Exp
```

Since the short circuiting evaluates from left to right order of the `Exp`, changing the order of the left and right subtrees would affect the performance of the traversal. As it can be seen in table 2, the layout where the left subtree is serialized before the right subtree results in better performance compared to the tree where the the right subtree is serialized before the left one. This is as expected since in the latter case, the traversal has to jump over the right subtree serialized before the left one in order to evaluate it first and then depending on the result of the left subtree possibly jump back to evaluate the right subtree. This results in poor spatial locality and hence worse performance. As the table shows MARMOSET is able to identify the layout transformations that would give the best performance and matches the case where the left subtree is serialized before the right one.

5.3 Blog Software Case Study

The Blog software case study serves as an example of a realistic benchmark, representing a sample of components from a blog management web-service. The main data structure is a linked-list of blogs where each blog contains various fields such as header, id, author information, content, hashtags and date. The fields are a mix of recursive and non-recursive datatypes. For instance, Content is a recursive type (a linked-list of strings), but Author is just a single string wrapped in a data constructor. Below we show one possible permutation of fields contained in each blog.

```
data Blogs = HIADCTB Header Id Author Date Content HashTags Blogs
            | Empty
```

We evaluate MARMOSET's performance using three different traversals over a list of blogs. Overall, these traversals only use three fields: Content and Tags, and the tail of the linked-list, Blogs. The fields used by each individual traversal as referred as *active fields* and the rest are referred as *passive fields*, and we specify these per-traversal in the following. In Table 3 we report the performance of all six layouts obtained by permuting the order of these three fields, and two additional layouts (Columns 1 and 2). The column names are indicative of the order of fields used. For example, the column `hiadctb` reports numbers for a layout where the fields are ordered as: Header, Id, Author, Date, Content, HashTags, and Blogs (the tail of the list). We use GIBBON to gather these run times. The last column shows the run times of these traversals using MARMOSET. The layout it picks is tailored to each individual traversal so we specify it per-traversal next.

	<i>hiadctb</i>	<i>ctbhiad</i>	<i>tbchiad</i>	<i>tcbhiad</i>	<i>btchiad</i>	<i>bchiadt</i>	<i>cbiadht</i>	MARMOSSET
<i>FilterBlogs</i>								
<i>Mean</i>	0.216	0.203	0.064	0.248	0.281	0.277	0.205	0.065
<i>Median</i>	0.216	0.203	0.064	0.248	0.281	0.278	0.205	0.065
<i>CI 95%</i>								
<i>ub</i>	0.216	0.204	0.065	0.251	0.283	0.279	0.206	0.065
<i>lb</i>	0.215	0.215	0.063	0.246	0.279	0.276	0.204	0.066
<i>EmphContent</i>								
<i>Mean</i>	0.800	0.768	1.076	1.073	1.073	1.064	0.624	0.637
<i>Median</i>	0.800	0.767	1.077	0.802	1.073	1.063	0.625	0.637
<i>CI 95%</i>								
<i>ub</i>	0.803	0.771	1.082	0.809	1.075	1.068	0.626	0.635
<i>lb</i>	0.797	0.764	1.071	0.799	1.070	1.061	0.622	0.639
<i>TagSearch</i>								
<i>Mean</i>	2.325	2.324	2.560	2.231	2.593	2.624	2.264	2.222
<i>Median</i>	2.328	2.322	2.560	2.228	2.591	2.621	2.264	2.216
<i>CI 95%</i>								
<i>ub</i>	2.337	2.328	2.562	2.235	2.596	2.628	2.264	2.211
<i>lb</i>	2.313	2.320	2.558	2.227	2.589	2.619	2.263	2.233

Table 3. Runtimes: Mean, Median and 95% confidence interval in seconds for different traversals for a subset permutation of different layouts when compiled with GIBBON. The last column shows the runtime of the layout chosen by MARMOSSET. Legend: h - Header, t - HashTags, b - Blogs, i - TagID, c - Content, a - Author, d - Date. The numbers in Blue correspond to the lowest running time and the numbers in Red correspond to the highest running time.

5.3.1 filterBlogs. This traversal filters the list of blogs and only retains those which contain the given hash-tag in their HashTags field. Since Blogs, Content, and HashTags are recursive fields, changing their layout should represent greater differences in performance. The *active fields* for this traversal are HashTags and Blogs. Theoretically, the performance of this traversal is optimized when the HashTags field is serialized before Blogs on account of the first access to HashTags in the traversal. This is confirmed in practice with the layout *tbchiad* being the fastest. The access to HashTags comes first since its serialized first in memory and once HashTags is fully traversed, the traversal can access blogs in an optimal manner without unnecessary movement in the packed buffer.

Indeed, the layout chosen by MARMOSSET puts HashTags first followed by Blogs. The order of the rest of the fields remains unchanged compared to the source program, but this has no effect on performance since they are *passive fields*. As shown in Table 3, the performance of the layout chosen by MARMOSSET is similar to the layout *tbchiad* when compiled with GIBBON.

5.3.2 emphContent. This traversal takes a keyword and a list of blogs as input. It then searches the content of each blog for the the keyword and emphasizes all its occurrences if present. If the keyword isn't present, the content remains unchanged. The active fields in this traversal are Content and Blogs. Therefore, based on the access pattern (Content accessed before Blogs), the layout with the best performance should place Content first followed by Blogs. Indeed, the layout with the best

	MARMOSET (<i>without pragma</i>)	MARMOSET (<i>with pragma</i>)
<i>TagSearch</i>		
<i>Then</i>	2.228	2.228
<i>TagSearch</i>		
<i>Else</i>	0.104	0.0367

Table 4. Performance of traversal 3 (Tag Search) when the input is biased for either the “Then” or “Else” branch through the traversal. MARMOSET (without pragma) shows the run-time for the layout chosen by MARMOSET without user supplied pragmas and MARMOSET (with pragma) shows the run-time when the user supplies likelihood values through pragmas in branches. The layout chosen in both the cases (with and without pragma is the same). Legend: h - Header, t - HashTags, b - Blogs, i - TagID, c - Content, a - Author, d - Date

performance when compiled with GIBBON is ***cbiadht***. The layout that MARMOSET chooses also prioritizes the placement of Content before Blogs. The other passive fields are all placed afterwards. As shown in table 3, the performance of the layout chosen by MARMOSET is similar to ***cbiadht***.

5.3.3 tagSearch. This traversal takes a keyword and a list of blogs as input. It looks for the presence of a keyword in the HashTags field. If it is present, it emphasizes this keyword in the Content, and leaves it unchanged otherwise. The layout that gives the best performance is ***tcbhiad*** when compiled with GIBBON. This is as expected based on the access patterns of the traversal which access HashTags followed by Content followed by Blogs. The layout MARMOSET chooses also places these fields in the same order. The performance as shown in Table 3 and is similar to ***tcbhiad***.

5.3.4 tagSearch: varying likelihood of paths taken. TagSearch’s source code is similar to our running example 1 where we take different code paths based on if a keyword is present in the tags of the blog post or not. Thus, depending on which branch is more likely, different layouts of the Blog value can have different runtime performance.

This information cannot be inferred statically since the contents of the Blog is known only at runtime. This means that MARMOSET may not be able to generate an optimal layout on its own since it does not have the information about which branch is more likely via statically analyzing the structure of the program. (As mentioned in Section 3, we assume uniform likelihoods for branches if no information is provided.)

As a case study, we show the performance of the Blog data type for an input which biases both the “Then” and “Else” branch of the traversal separately. Without information on which branch is likely, MARMOSET produces a layout which has sub-optimal performance in the “Else” branch. The performance of this layout is shown in Table 4 in the MARMOSET (without pragma) column. This layout places HashTags first, followed by Content, Blogs and then the passive fields. Since the traversal goes to the “Else” branch, it has to jump over the Content fields serialized in between HashTags and Blogs resulting in poor performance.

When MARMOSET is given the information about which branch is likely, it generates a different layout where it places the HashTags field first, followed by Blogs, Content and finally the passive fields. This gives better performance as shown in the column MARMOSET (with pragma) of table 4. Since branch frequency from profiling data is a reasonable assumption, MARMOSET can generate more optimal layouts when compared to GIBBON. We also show the performance of the optimal layout generated by MARMOSET for the “Then” branch in Figure 4.

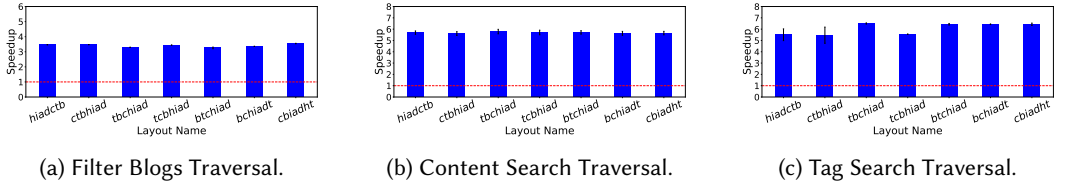


Fig. 6. Speedups: Median and 95% confidence interval speedups of MARMOSSET over GHC for different traversals for a subset permutation of different layouts when compiled with GHC (Strict Evaluation). Legend: h - Header, t - HashTags, b - Blogs, i - TagID, c - Content, a - Author, d - Date.

	<i>hiaadctb</i>	<i>ctbhiad</i>	<i>tbchiad</i>	<i>tcbhiad</i>	<i>btchiad</i>	<i>bchiadt</i>	<i>cbiadht</i>	MARMOSSET
<i>FilterBlogs</i>								
<i>L3 misses</i>	1.06e7	1.03e7	1.31e6	1.03e7	1.1e7	1.11e7	5.40e6	1.47e6
<i>L3 accesses</i>	1.34e7	1.27e7	6.97e6	1.33e7	1.30e7	1.30e7	1.13e7	7.51e6
<i>L3 miss rate</i>	0.788	0.808	0.188	0.776	0.844	0.850	0.475	0.196
<i>Instructions</i>	3.89e8	3.91e8	3.91e8	3.89e8	3.88e8	3.88e8	3.91e8	3.91e8
<i>ContentSearch</i>								
<i>L3 misses</i>	1.03e7	9.36e6	1.60e7	9.31e6	1.64e7	1.67e7	8.15e6	8.16e6
<i>L3 accesses</i>	3.00e7	2.97e7	3.27e7	3.01e7	3.25e7	3.28e7	2.86e7	2.85e7
<i>L3 miss rate</i>	0.343	0.315	0.489	0.309	0.504	0.509	0.285	0.286
<i>Instructions</i>	6.24e9	6.24e9	6.71e9	6.24e9	6.71e9	6.71e9	6.24e9	6.24e9
<i>TagSearch</i>								
<i>L3 misses</i>	3.23e7	3.18e7	3.19e7	2.61e7	3.29e7	3.45e7	2.79e7	2.54e7
<i>L3 accesses</i>	1.11e8	1.11e8	1.19e8	1.07e8	1.21e8	1.21e8	1.35e8	1.07e8
<i>L3 miss rate</i>	0.291	0.286	0.268	0.244	0.271	0.284	0.207	0.238
<i>Instructions</i>	2.44e10	2.44e10	2.48e10	2.44e10	2.48e10	2.48e10	2.44e10	2.44e10

Table 5. Various performance stats (*PAPI*) for different traversals for a subset permutation of different layouts when compiled with GIBBON. The last column shows the cache misses for the layout chosen by MARMOSSET. Legend: h - Header, t - HashTags, b - Blogs, i - TagID, c - Content, a - Author, d - Date. The numbers in Blue correspond to the lowest cache misses and the numbers in Red correspond to the highest cache misses.

5.3.5 Comparison of MARMOSSET against GIBBON and GHC. We compare MARMOSSET's performance to GIBBON, and also GHC, which is especially optimized to run functional programs.

Figure 6 shows the the speedup of MARMOSSET over GHC when evaluated strictly for an apples to apples comparison. We pass the optimization option `-O2` to GHC while compiling. As it can be seen the performance of MARMOSSET is better than GHC by significant margins for all the layouts and traversals. Since the types in GHC are *boxed* this is expected since it adds more instructions and results in worse spatial locality.

5.4 Cache behavior

The results from earlier sections demonstrate that MARMOSSET's layout choices do, indeed, improve performance. This section investigates *why* performance improves. The basic premise of MARMOSSET's approach to layout optimization is to concentrate on minimizing how often a traversal

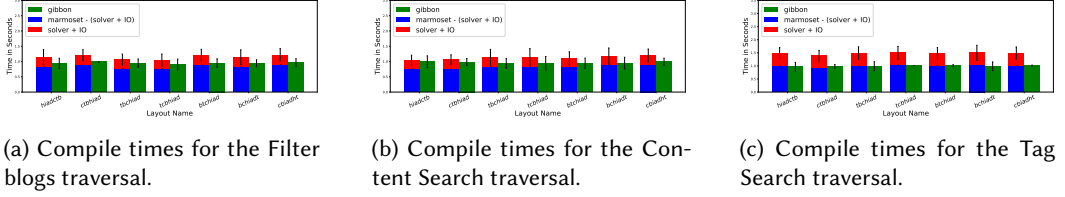


Fig. 7. Average compile times (9 runs) in seconds for different layouts and traversal combinations when compiled with GIBBON and when optimized for layout by MARMOSET. The bar in *Blue* is the compile time in seconds for MARMOSET after subtracting the time for solver IO and the bar in *Red* is the time taken to do the IO call to the DOCPLEX solver. Note that the order of active fields chosen in the optimized layout for a particular traversal by MARMOSET is the same for all the initial layouts. The total compile time for MARMOSET includes the time for doing the CFG analysis, generating the access graph, the solver time, reordering the data type and repairing the code, The majority of which is added by the solver IO.

needs to backtrack or skip ahead while processing a buffer. By minimizing this jumping around, we expect to see improvements from two possible sources. First, we expect to see an improvement in instruction counts, as an optimized layout should do less pointer chasing. Second, we expect to see an improvement in L3 cache utilization: both fewer misses (due to improved spatial locality and prefetching) and fewer accesses (due to improved locality in higher level caches).

To substantiate this hypothesis, we collected performance counter measurements for various layouts. The results are shown in Table 5.⁶ We see that our main hypothesis is borne out. The optimal layout indeed has far fewer L3 accesses and L3 misses: the better layout promotes better locality. Interestingly, we do not observe a similar effect for instruction count. While different layouts do differ somewhat in instructions, the difference is slight. We suspect this is partially due to GIBBON's current implementation, that often dereferences pointers even if a simple access in the buffer would suffice.

5.5 Compile times of the different layouts when compiled with GIBBON and MARMOSET

We show the compile times in figure 7 for different layout and traversal combinations when compiled with GIBBON and when optimized with MARMOSET respectively. The compile times for MARMOSET include the time to generate the control flow graph, the field access graph, the solver time, the time to re-order the data type and finally repairing the code so that is typechecks. The solver times are in the order of the number of fields in a data constructor and not the program size. Hence, the solver adds relatively low overhead. Since the compiler does an IO call to the python solver, there is room for improvement in the future to lower these times. For instance, we could directly perform *FFI* calls to the CPLEX solver by lowering the constraints to C code in MARMOSET. This would be faster and safer than the current implementation.

6 RELATED WORK

6.1 Cache-conscious data

Using generational garbage collection to implement cache-conscious data placement [Chilimbi and Larus 1998]. This work uses cache conscious data placement in tandem with a generational copying garbage collector. The idea is that in a scavenge phase of the garbage collector, the copying

⁶Due to a configuration issue, we were unable to obtain performance counter results for our primary experimental platform. These results were collected on a 32-core, 2.7 GHz machine with 20MB LLC. We hope to have performance counter results for our primary machine for the next version.

algorithm should be augmented such that it places the objects in the “TO” space such that it results in better cache performance. There are two properties of object oriented programming that are utilized. Objects are less than 32 KB and that they are not lightweight use objects. Since the cache block sizes these days are 64 KB this means that the fields of the objects need not be optimized for and the only layout optimization should be object level optimization. The paper introduces a concepts of a object affinity graph which is a graph consisting of vertices as live objects and edges encode the temporal affinity between the objects. This graph is generated by hooking the application with a real time profiler that tracks these temporal affinities. During a scavenger phase the copying garbage collector then copies these objects over to the “TO” space in an order that increases the cache performance. They use a greedy algorithm that traverses the max weighted edge and copies the objects over in this greedy fashion. Miss rates are improved by 20% to 41% and performance by 18% to 31% compared to prior work.

Cache-Conscious Structure Definition [Chilimbi et al. 1999a]. The work extends the work Cache-Conscious structure layout by focusing on data structures that are larger than the size of a cache block. Previous work worked best for data structures that are around half the size of a cache block. This work builds upon the prior work by extending it with two more techniques. These are **structure splitting** and **field reordering**. Mapping concurrently accessed elements of a struct in the same cache block provides an implicit prefetch. If these elements do not fit in the same cache block then placing them in non-conflicting cache blocks will reduce cache misses. This paper introduces the idea of hot cold splitting of a data structure, categorizing elements as being “hot” if accessed frequently and “cold” if accessed infrequently. This information is obtained by profiling the program, cold fields are placed into a new object via an indirection and hot fields remain unchanged. At runtime, the cache-conscious garbage collector [Chilimbi and Larus 1998] co-locates the modified object instances. This paper also suggests placing fields with high temporal affinity within the same cache block. For this they recommend bbcache, a field recommender for a data structure. bbcache forms a field affinity graph which combines static information about the source location of structure field accesses with dynamic information about the temporal ordering of accesses and their access frequency.

Cache-Conscious Structure Layout [Chilimbi et al. 1999b]. Poor reference locality is one major bottleneck in today’s programs. Since programmers often do not program keeping in mind the hierarchical structure of caches in today’s architectures, the memory performance of these programs is sometimes far from ideal. Their proposed technique improves on prior work in that it requires less programmer effort, less architectural dependence and less source code modifications. They propose two techniques to solve the problem of poor reference locality in programs.

- **ccmorph:** This works on tree-like data structures, and it relies on the programmer making a calculated guess about the safety of the operation on the tree-like data structure. It performs two major optimizations: clustering and coloring. Clustering takes a tree-like data structure and attempts to pack likely to be accessed elements in the structure within the same cache block. There are various ways to pack a subtree, including clustering k nodes in a subtree together, depth first clustering, etc. Coloring attempts to map simultaneously accessed data elements to non-conflicting regions of the cache.
- **ccmalloc:** This is a memory allocator similar to malloc which takes an additional parameter that points to an existing data structure element which is likely accessed simultaneously. This requires programmer knowledge and effort in recognizing and then modifying the code with such a data element. ccmalloc tries to allocate the new data element as close to the existing data as possible, with the initial attempt being to allocate in the same cache

block. It tries to put likely accessed elements on the same page in an attempt to improve TLB performance.

Using these techniques they are able to get significant improvements in the run-time of the program. Future work is to get rid of programmer effort and be able to analyze the program statically or profiling techniques.

SHAPES [Franco et al. 2017, 2019]. describes an effort to attain good cache performance without sacrificing abstraction. The vision is that the layout of a data structure should be defined once at the point of initialization, and all further code that interacts with the structure should be “layout agnostic”. Ideally, this means that performance improvements involving layout changes can be made without requiring changes to program logic. To achieve this, classes are extended to support different layouts, and types carry layout information—code that operates on objects may be polymorphic over the layout details.

6.2 Data layout description and binary formats

Dargent [Chen et al. 2023]. is a data layout description framework that allows programmers to specify the data layout of an algebraic data type. It is built on top of the Cogent language [O'Connor et al. 2016], which is a first order polymorphic functional programming language. *Dargent* targets C code and provides proofs of formal correctness of the compiled C code with respect to the layout descriptions. Rather than having a compiler attempt to determine an efficient layout, their focus is on allowing the programmer to specify a particular layout they want and have confidence in the resulting C code.

Additionally, there has been significant prior work on the generation of verified, efficient code for interacting with binary data formats (especially parsing and validating). For example, *Ever-Parse* [Ramananandro et al. 2019] is a framework for generating verified parsers and formatters for binary data formats, and it has been used to formally verify zero-copy parsers for authenticated message formats. With *Narcissus* [Delaware et al. 2019], encoders and decoders for binary formats could be verified and extracted, allowing researchers to certify the packet processing for a full Internet protocol stack. Other work [van Geest and Swierstra 2017] has also explored the automatic generation of verified parsers and pretty printers given a specification of a binary data format, as well as the formal verification of a compiler for a subset of the Protocol Buffer serialization format [Ye and Delaware 2019].

Prior to that, there was earlier work [Back 2002] that demonstrated how a domain-specific language for describing binary data formats could be useful for generating validators and for easier scripting and manipulation of the data from a high-level language like Java. In other earlier related work, *Packet Types* [McCann and Chandra 2000] was an attempt at using types for programming with network protocol messages, with language-level support for features commonly found in protocol formats like variable-sized and optional fields.

6.3 Memory layouts

Floorplan [Cronburg and Guyer 2019]. is a declarative language for specifying high-level memory layouts, implemented as a compiler which generates Rust code. The language has forms for specifying sizes, alignments, and other features of chunks of memory in the heap, with the idea that any correct state of the heap can be derived from the *Floorplan* specification. It was successfully used to eliminate 55 out of 63 unsafe lines of code (all of the unsafe code relating to memory safety) of the *immix* garbage collector. Related earlier work is *PADS* [Fisher and Gruber 2005; Fisher and Walker 2011; Mandelbaum et al. 2007], a language for describing ad hoc data file formats.

Automatic Pool Allocation for Disjoint Data Structures [Lattner and Adve 2002]. This paper introduces a technique to perform data layout optimization at link time during compilation. The idea is that it is easier to perform intraprocedural optimization at link time when multiple files are being compiled together. The compiler generates data structure graphs of disjoint data structures for each functions, for data structures that are accessed in a type safe manner, it automatically allocates pools for data structures.

7 CONCLUSIONS

This paper introduced MARMOSSET, which builds on Gibbon to generate efficient orders for Algebraic Data Types. We observed that a fairly straightforward control flow analysis allows MARMOSSET to identify opportunities to place fields of a data constructor near each other in memory to promote efficient consecutive access to those fields. Because a given function might use many fields in many different ways, MARMOSSET adopts an approach of formulating the data layout problem as an ILP, with a cost model that assigns an abstract cost to a chosen layout. This ILP can be further constrained with user-provided layout guidelines. Armed with problem formulation, an off-the-shelf ILP solver allows MARMOSSET to generate minimal-(abstract)-cost layouts for algebraic data types. MARMOSSET then uses these layouts to synthesize a new ADT and prior work's Gibbon compiler toolchain to lower the code into an efficient program that operates over packed data types with minimal pointer chasing.

We found, across a number of benchmarks, that MARMOSSET is able to effectively and consistently find the optimal data layout for a given combination of traversal function and ADT. We showed, experimentally, that MARMOSSET-optimized layouts significantly outperform not only Gibbon's default layouts, but also GHC's standard compiler.

REFERENCES

- Accessed: 2023. IBM® Decision Optimization CPLEX® Modeling for Python. <https://ibmdecisionoptimization.github.io/docplex-doc/>.
- Godmar Back. 2002. DataScript - A Specification and Scripting Language for Binary Data. In *Proceedings of the 1st ACM SIGPLAN/SIGSOFT Conference on Generative Programming and Component Engineering (GPCE '02)*. Springer-Verlag, Berlin, Heidelberg, 66–77.
- Zilin Chen, Ambrose Lafont, Liam O'Connor, Gabriele Keller, Craig McLaughlin, Vincent Jackson, and Christine Rizkallah. 2023. Dargent: A Silver Bullet for Verified Data Layout Refinement. *Proc. ACM Program. Lang.* 7, POPL, Article 47 (jan 2023), 27 pages. <https://doi.org/10.1145/3571240>
- Trishul M. Chilibi, Bob Davidson, and James R. Larus. 1999a. Cache-Conscious Structure Definition. In *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation* (Atlanta, Georgia, USA) (PLDI '99). Association for Computing Machinery, New York, NY, USA, 13–24. <https://doi.org/10.1145/301618.301635>
- Trishul M. Chilibi, Mark D. Hill, and James R. Larus. 1999b. Cache-Conscious Structure Layout. In *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation* (Atlanta, Georgia, USA) (PLDI '99). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/301618.301633>
- Trishul M. Chilibi and James R. Larus. 1998. Using Generational Garbage Collection to Implement Cache-Conscious Data Placement. In *Proceedings of the 1st International Symposium on Memory Management* (Vancouver, British Columbia, Canada) (ISMM '98). Association for Computing Machinery, New York, NY, USA, 37–48. <https://doi.org/10.1145/286860.286865>
- Adam Chlipala. 2015. An Optimizing Compiler for a Purely Functional Web-application Language. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming* (Vancouver, BC, Canada) (ICFP 2015). ACM, New York, NY, USA, 10–21. <https://doi.org/10.1145/2784731.2784741>
- Karl Cronburg and Samuel Z. Guyer. 2019. Floorplan: Spatial Layout in Memory Management Systems. In *Proceedings of the 18th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences* (Athens, Greece) (GPCE 2019). Association for Computing Machinery, New York, NY, USA, 81–93. <https://doi.org/10.1145/3357765.3359519>
- Benjamin Delaware, Sorawit Suriyakarn, Clément Pit-Claudel, Qianchuan Ye, and Adam Chlipala. 2019. Narcissus: Correct-by-Construction Derivation of Decoders and Encoders from Binary Formats. *Proc. ACM Program. Lang.* 3, ICFP, Article 82 (jul 2019), 29 pages. <https://doi.org/10.1145/3341686>

- Kathleen Fisher and Robert Gruber. 2005. PADS: A Domain-Specific Language for Processing Ad Hoc Data. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (Chicago, IL, USA) (PLDI '05). Association for Computing Machinery, New York, NY, USA, 295–304. <https://doi.org/10.1145/1065010.1065046>
- Kathleen Fisher and David Walker. 2011. The PADS Project: An Overview. In *Proceedings of the 14th International Conference on Database Theory* (Uppsala, Sweden) (ICDT '11). Association for Computing Machinery, New York, NY, USA, 11–17. <https://doi.org/10.1145/1938551.1938556>
- Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. The Essence of Compiling with Continuations. In *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation* (Albuquerque, New Mexico, USA) (PLDI '93). Association for Computing Machinery, New York, NY, USA, 237–247. <https://doi.org/10.1145/155090.155113>
- Juliana Franco, Martin Hagelin, Tobias Wrigstad, Sophia Drossopoulou, and Susan Eisenbach. 2017. You Can Have It All: Abstraction and Good Cache Performance. In *Proceedings of the 2017 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Vancouver, BC, Canada) (Onward! 2017). Association for Computing Machinery, New York, NY, USA, 148–167. <https://doi.org/10.1145/3133850.3133861>
- Juliana Franco, Alexandros Tasos, Sophia Drossopoulou, Tobias Wrigstad, and Susan Eisenbach. 2019. Safely Abstracting Memory Layouts. <https://doi.org/10.48550/ARXIV.1901.08006>
- Chris Lattner and Vikram Adve. 2002. Automatic Pool Allocation for Disjoint Data Structures. In *Proceedings of the 2002 Workshop on Memory System Performance* (Berlin, Germany) (MSP '02). Association for Computing Machinery, New York, NY, USA, 13–24. <https://doi.org/10.1145/773146.773041>
- Yitzhak Mandelbaum, Kathleen Fisher, David Walker, Mary Fernandez, and Artem Gleyzer. 2007. PADS/ML: A Functional Data Description Language. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Nice, France) (POPL '07). Association for Computing Machinery, New York, NY, USA, 77–83. <https://doi.org/10.1145/1190216.1190231>
- Peter J. McCann and Satish Chandra. 2000. Packet Types: Abstract Specification of Network Protocol Messages. In *Proceedings of the Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication* (Stockholm, Sweden) (SIGCOMM '00). Association for Computing Machinery, New York, NY, USA, 321–333. <https://doi.org/10.1145/347059.347563>
- Liam O'Connor, Christine Rizkallah, Zilin Chen, Sidney Amani, Japheth Lim, Yutaka Nagashima, Thomas Sewell, Alex Hixon, Gabriele Keller, Toby Murray, et al. 2016. COGENT: certified compilation for a functional systems language. *arXiv preprint arXiv:1601.05520* (2016).
- Tahina Ramananandro, Antoine Delignat-Lavaud, Cédric Fournet, Nikhil Swamy, Tej Chajed, Nadim Kobeissi, and Jonathan Protzenko. 2019. Everparse: Verified Secure Zero-Copy Parsers for Authenticated Message Formats. In *Proceedings of the 28th USENIX Conference on Security Symposium* (Santa Clara, CA, USA) (SEC'19). USENIX Association, USA, 1465–1482.
- D. N. Truong, F. Bodin, and A. Sez nec. 1998. Improving Cache Behavior of Dynamically Allocated Data Structures. In *Proceedings of the 1998 International Conference on Parallel Architectures and Compilation Techniques* (PACT '98). IEEE Computer Society, USA, 322.
- Marcell van Geest and Wouter Swierstra. 2017. Generic Packet Descriptions: Verified Parsing and Pretty Printing of Low-Level Data. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Type-Driven Development* (Oxford, UK) (TyDe 2017). Association for Computing Machinery, New York, NY, USA, 30–40. <https://doi.org/10.1145/3122975.3122979>
- Michael Vollmer, Chaitanya Koparkar, Mike Rainey, Laith Sakka, Milind Kulkarni, and Ryan R. Newton. 2019. LoCal: A Language for Programs Operating on Serialized Data. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 48–62. <https://doi.org/10.1145/3314221.3314631>
- Michael Vollmer, Sarah Spall, Buddhika Chamith, Laith Sakka, Chaitanya Koparkar, Milind Kulkarni, Sam Tobin-Hochstadt, and Ryan R. Newton. 2017. Compiling Tree Transforms to Operate on Packed Representations. In *31st European Conference on Object-Oriented Programming (ECOOP 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 74)*, Peter Müller (Ed.). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 26:1–26:29. <https://doi.org/10.4230/LIPIcs.ECOOP.2017.26>
- Qianchuan Ye and Benjamin Delaware. 2019. A Verified Protocol Buffer Compiler. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Cascais, Portugal) (CPP 2019). Association for Computing Machinery, New York, NY, USA, 222–233. <https://doi.org/10.1145/3293880.3294105>